

Q3: API Endpoints, Routing Structure & Global Middleware

***Question:** Outline the main API endpoints and how routing is structured. What are the core groups of endpoints? Point out any global dependencies, middleware, or authentication mechanisms applied before a request hits core logic.*

How Routing Is Structured

All routers are registered in `app/main.py` under the `/api/v1/` prefix. There are 12 routers, each living in its own file under `app/api/v1/`. The pattern is:

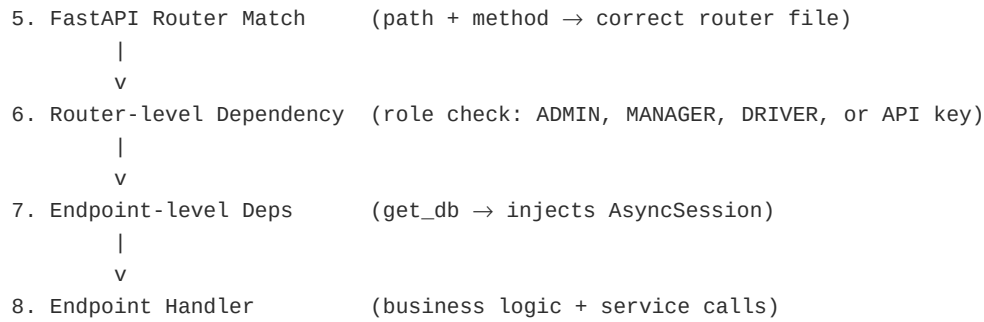
```
app/main.py
app.include_router(auth.router,      prefix="/api/v1/auth")
app.include_router(hardware.router,  prefix="/api/v1/hardware")
app.include_router(manager_ops.router, prefix="/api/v1/manager")
app.include_router(admin.router,     prefix="/api/v1/admin")
app.include_router(users.router,     prefix="/api/v1/admin/users")
app.include_router(drivers.router,   prefix="/api/v1/admin/drivers")
app.include_router(vehicles.router,  prefix="/api/v1/admin/vehicles")
app.include_router(routes.router,    prefix="/api/v1/admin/routes")
app.include_router(tickets.router,   prefix="/api/v1/admin/tickets")
app.include_router(manager.router,   prefix="/api/v1/manager")
app.include_router(driver.router,    prefix="/api/v1/drivers")
app.include_router(websocket.router) # mounts at /ws
```

Role enforcement is applied at the **router level**, not per-endpoint, using `APIRouter(dependencies=[...])`. This means every route inside that router automatically inherits the auth/role check without repeating it.

The Full Request Pipeline

Before any endpoint logic runs, a request passes through this ordered pipeline:

```
Incoming HTTP Request
|
v
1. SanitizationMiddleware (ASGI — strips XSS from JSON bodies)
|
v
2. IdempotencyMiddleware (ASGI — blocks duplicate POST/PUT/PATCH via Redis key)
|
v
3. CORSMiddleware (checks Origin header against BACKEND_CORS_ORIGINS)
|
v
4. slowapi RateLimiter (100 req/min per IP globally; tighter limits per endpoint)
|
v
```



Layer 1: SanitizationMiddleware

File: `app/middleware/sanitization.py`

Type: Pure ASGI middleware (not Starlette `BaseHTTPMiddleware`)

Intercepts the raw byte stream **before** FastAPI reads the body. Applies to all `POST` , `PUT` , `PATCH` requests with `Content-Type: application/json` .

- Collects all body chunks into a single buffer
- Parses as JSON, recursively walks all string values
- Runs `bleach.clean(value, tags=[], strip=True)` on every string — removes all HTML tags
- Re-serializes to JSON and replaces the request body in-place
- Strips `Content-Length` header (to avoid mismatch after sanitization)

This runs before any Pydantic validation, so it is not possible for HTML/script content to reach the ORM layer.

Layer 2: IdempotencyMiddleware

File: `app/middleware/idempotency.py`

Type: Starlette `BaseHTTPMiddleware`

Prevents duplicate mutations when clients retry requests.

- Only active when client sends `Idempotency-Key: <uuid>` header
 - Only applies to `POST` , `PUT` , `PATCH`
 - Key stored in Redis: `idempotency:{METHOD}:{path}:{key}` with 24-hour TTL using `SETNX`
 - If the key already exists: returns `HTTP 409` immediately
 - If the upstream handler returns a 5xx: deletes the key from Redis (allows safe retry)
-

Layer 3: CORSMiddleware

Configured from `settings.BACKEND_CORS_ORIGINS` (read from `.env`). Default allowed origins:

```
http://localhost:5173
http://localhost:5174
http://localhost:3000
```

All methods and headers are permitted (`allow_methods=["*"]` , `allow_headers=["*"]`). Credentials are allowed (`allow_credentials=True`).

Layer 4: Rate Limiting (slowapi)

File: `app/core/rate_limit.py`

```
limiter = Limiter(key_func=get_remote_address, default_limits=["100/minute"])
```

Global default: **100 requests/minute per IP**. Individual endpoints override this with stricter limits:

Endpoint	Limit
POST <code>/auth/login</code>	5/minute
POST <code>/hardware/gps</code>	120/minute
POST <code>/hardware/iot</code>	60/minute
POST <code>/hardware/anpr</code>	60/minute
POST <code>/hardware/camera</code>	60/minute

Exceeding the limit returns `HTTP 429 Too Many Requests` .

Layer 5–6: Authentication & Role Enforcement

File: `app/api/deps.py`

Two authentication mechanisms exist in parallel:

A. JWT Bearer Token (human users)

All human-facing routers use this chain:

```
# Step 1: Extract and decode JWT
get_current_user(token, db)
    → jwt.decode(token, SECRET_KEY)
    → SELECT * FROM users WHERE email = payload["sub"]

# Step 2: Check is_active
get_current_active_user(current_user)
    → if not current_user.is_active: raise 400

# Step 3: Check role
get_current_user_with_role(UserRole.ADMIN)
```

```
→ if current_user.role not in required_roles: raise 403
```

Tokens are standard JWT (HS256). The `role` claim is embedded in the token payload at login time so the role check does not need an extra DB query.

Token blacklist: When a user changes their password, the current access token is stored in Redis under `blacklist:{token}` with the remaining TTL. Blacklist checking must be implemented in `get_current_user` — currently the token is blacklisted in Redis but the check is done gracefully (Redis-optional).

B. Hardware API Key (IoT devices)

The `hardware` router uses a completely separate auth mechanism — no JWT involved:

```
async def verify_hardware_api_key(x_hardware_api_key: Optional[str] = Header(None)):
    if not secrets.compare_digest(key, settings.HARDWARE_API_KEY):
        raise HTTPException(403, "Invalid or missing hardware API key")

router = APIRouter(dependencies=[Depends(verify_hardware_api_key)])
```

`secrets.compare_digest` is used (constant-time comparison) to prevent timing attacks. The key is passed via the `X-Hardware-API-Key` request header.

Endpoint Groups

Group 1: Authentication — `/api/v1/auth`

Auth required: None for login/refresh. JWT required for `/me`, `/me` PATCH, and `/change-password`. ADMIN JWT required for `/signup`.

Method	Path	Auth	Purpose
POST	<code>/login</code>	None	OAuth2 form login. Returns access + refresh tokens. Rate-limited: 5/min
POST	<code>/refresh</code>	None	Exchange refresh token (Redis lookup) for new access token
POST	<code>/signup</code>	ADMIN JWT	Create a new user (any role)
GET	<code>/me</code>	JWT	Return current user's profile
PATCH	<code>/me</code>	JWT	Update name, phone, preferred language
POST	<code>/change-password</code>	JWT	Validate old password, set new, blacklist current token

Group 2: Admin — `/api/v1/admin`

Router-level auth:

```
APIRouter(dependencies=[Depends(get_current_user_with_role(UserRole.ADMIN))])
```

Every endpoint in this group automatically requires a valid ADMIN JWT.

Dashboard

Method	Path	Purpose
GET	/admin/dashboard/stats	Counts: vehicles, drivers, routes, users, pending maintenance, active trips, trips-per-route chart data

Schedule & Rotations

Method	Path	Purpose
POST	/admin/rotations	Manually create a single rotation assignment
POST	/admin/rotations/generate	Trigger daily schedule generation. Pre-flight validates: active routes exist, ≥3 OFF_DUTY drivers, ≥2 FREE vehicles

Trips

Method	Path	Purpose
GET	/admin/trips	All trips for a given date (default: today), with route + driver + vehicle names
GET	/admin/trips/{id}	Full trip detail: route, driver, vehicle, tickets

Maintenance

Method	Path	Purpose
GET	/admin/maintenance	All maintenance requests, optional <code>?status=</code> filter

Reporting

Method	Path	Purpose
GET	/admin/reports/daily	Aggregate stats for a given date

Method	Path	Purpose
GET	/admin/reports/query	Filtered report by date range, driver, route
GET	/admin/reports/breakdown	Revenue + trip counts broken down by route and shift
GET	/admin/reports/export	Download as PDF or CSV (?format=pdf\ csv&start=&end=)

Audit Logs

Method	Path	Purpose
GET	/admin/audit-logs	Paginated audit trail (?skip=&limit=)

Group 3: Admin Sub-Resources — CRUD Routers

Each is a standalone file with `APIRouter(dependencies=[ADMIN])` . Mounted under `/api/v1/admin/` :

Users — `/api/v1/admin/users``

Method	Path	Purpose
GET	/	List all users (paginated)
POST	/	Create user
PUT	/ {id}	Update user (name, phone, language, active status, password)
DELETE	/ {id}	Soft-delete (sets <code>is_active=False</code>)
POST	/with-driver	Atomic create: User + Driver profile in one transaction

Drivers — `/api/v1/admin/drivers``

Method	Path	Purpose
GET	/	List all driver profiles
GET	/ {id}	Single driver detail
POST	/	Create driver profile
PUT	/ {id}	Update driver (license, status, vehicle assignment)

Method	Path	Purpose
DELETE	/ {id}	Remove driver

Vehicles — `/api/v1/admin/vehicles`

Method	Path	Purpose
GET	/	List all vehicles
GET	/ {id}	Single vehicle detail
POST	/	Add vehicle
PUT	/ {id}	Update (status, fuel, mileage, coordinates)
DELETE	/ {id}	Remove vehicle

Routes — `/api/v1/admin/routes`

Method	Path	Purpose
GET	/	List all routes (with stops embedded)
GET	/ {id}	Single route + stops
POST	/	Create route + stops in one call
PUT	/ {id}	Update route metadata
DELETE	/ {id}	Remove route

Tickets — `/api/v1/admin/tickets`

Method	Path	Purpose
GET	/	List all tickets
GET	/ {id}	Single ticket
POST	/	Issue ticket (ticket_code auto-generated)
PUT	/ {id}	Update ticket status
DELETE	/ {id}	Remove ticket

Group 4: Manager Operations — `/api/v1/manager`

Router-level auth: MANAGER or ADMIN JWT required.

Split across two files (`manager_ops.py` + `manager.py`), both mounted at the same prefix.

Method	Path	Purpose
GET	<code>/tracking/live</code>	All vehicles with non-null GPS coordinates (for live map)
GET	<code>/trips/active</code>	All ACTIVE trips with crowding scores
GET	<code>/reroute</code>	List all PENDING reroute requests
PATCH	<code>/reroute/{id}/approve</code>	Approve reroute, notify driver via WebSocket
PATCH	<code>/reroute/{id}/reject</code>	Reject reroute, notify driver via WebSocket
GET	<code>/maintenance</code>	Pending maintenance requests
PATCH	<code>/maintenance/{id}/approve</code>	Approve maintenance request
PATCH	<code>/maintenance/{id}/reject</code>	Reject with mandatory reason
GET	<code>/notifications</code>	All notifications for manager's user

Group 5: Driver — `/api/v1/drivers`

Router-level auth: DRIVER JWT required.

All paths are prefixed with `/me/` — a driver can only see and act on their own data.

Profile & Shift

Method	Path	Purpose
GET	<code>/me</code>	Own driver profile + break counters
POST	<code>/me/check-in</code>	Start shift: sets ACTIVE, resets daily counters
POST	<code>/me/check-out</code>	End shift: sets OFF_DUTY, auto-cancels remaining SCHEDULED trips

Trip Lifecycle

Method	Path	Purpose
GET	<code>/me/trips</code>	Own trip history (paginated, newest first)
GET	<code>/me/trips/{id}</code>	Single trip detail (ownership verified)

Method	Path	Purpose
POST	/me/trips/{id}/start	Start trip: enforces check-in, no concurrent active trips, no earlier SCHEDULED trips, 15-min early barrier
POST	/me/trips/{id}/end	End trip: enforces min duration (50% of route estimate), aggregates fare from tickets, sets is_late flag

Break Management

Method	Path	Purpose
POST	/me/break/start	Start break (validated by break_service)
POST	/me/break/end	End break, returns updated driver profile
GET	/me/break/status	Current break state and time remaining
GET	/me/breaks	Break history for today

Reroute & Notifications

Method	Path	Purpose
POST	/me/reroute	Submit reroute request for active trip
GET	/me/notifications	All notifications, newest first
PATCH	/me/notifications/{id}/read	Mark single notification as READ

Group 6: Hardware Data Ingestion — `/api/v1/hardware``

Router-level auth: `X-Hardware-API-Key` header (constant-time compare). No JWT.

All endpoints return `204 No Content` on success (except `/anpr` which returns the decision).

Method	Path	Rate Limit	Payload	Action
POST	/gps	120/min	vehicle_id, lat, lng, speed, heading	Update vehicle position, log to <code>gps_tracking</code> . Speed >80 km/h → WebSocket alert to MANAGER

Method	Path	Rate Limit	Payload	Action
POST	/iot	60/min	vehicle_id, engine_temp_c, oil_pressure_psi, brake_pad_mm, battery_voltage	Persist all 4 readings. Threshold breach → auto-create EMERGENCY MaintenanceRequest , WebSocket alert to ADMIN + MANAGER
POST	/anpr	60/min	plate_number, confidence, gate_id	Confidence <0.85 → IGNORED. Match active vehicle → GRANTED. No match → DENIED. Log to gate_logs . Broadcast decision via WebSocket
POST	/camera	60/min	trip_id, passenger_count	Calculate crowding_score = count / capacity . ≥70% → log CrowdingEvent . ≥90% → auto-dispatch DRIVER_3, WebSocket alert

IoT Thresholds:

Sensor	Threshold	Alert
Engine Temp	> 105°C	CRITICAL
Oil Pressure	< 25 PSI	LOW
Brake Pad	< 3 mm	WORN
Battery Voltage	< 11.8 V	LOW

Group 7: WebSocket — `/ws`

Auth: JWT passed as query param (`/ws?token=<JWT>`) or via `Sec-WebSocket-Protocol: jwt.token.<JWT>` header.

The `ConnectionManager` decodes the JWT, extracts the `role` , and places the connection into the correct role bucket (`ADMIN` , `MANAGER` , or `DRIVER`).

Feature	Description
Heartbeat	Client sends "ping" or {"type": "ping"} → server replies {"type": "pong"}
Idle timeout	60-second inactivity timeout — disconnects unresponsive clients
Rate limit	Max 10 messages/second per connection
Auto-reconnect	Handled client-side (frontend WebSocketContext.jsx)
Message routing	Redis broadcast channel → all roles. alerts channel → role-targeted via data["role"] field

Event types pushed by the server:

Type	Sent To	Trigger
speed_alert	MANAGER	GPS speed > 80 km/h
iot_alert	MANAGER + ADMIN	IoT sensor threshold breach
gate_auth	MANAGER + ADMIN	ANPR GRANTED or DENIED event
crowding_alert	MANAGER	Camera crowding score ≥ 90%
reroute_decision	DRIVER	Manager approves/rejects reroute
pong	Sender	Heartbeat response

Health Endpoints

Method	Path	Auth	Purpose
GET	/api/v1/health	None	Returns {"status": "healthy", "version": "1.1"}
GET	/	None	Root health check with version

Summary: Auth Model Per Group

Router Group	Auth Mechanism	Role Required
/auth/login , /auth/refresh	None	—

Router Group	Auth Mechanism	Role Required
/auth/signup	JWT	ADMIN
/auth/me , /auth/change-password	JWT	Any active user
/admin/**	JWT	ADMIN
/manager/**	JWT	MANAGER or ADMIN
/drivers/**	JWT	DRIVER
/hardware/**	API Key Header	Hardware device
/ws	JWT (query param)	Any active user

Generated: 2026-04-16